

ICT Trading Bot: Implementation Summary

Binance USD-M futures · branch `claude/keen-meitner-hlq76o` · 4 Oct 2026

1. Status

The original `ict_fvg_bot.py` was not in the repository, so the bot was written from scratch following the handoff spec. Standard library only. Unit tests pass (5/5). It has **not** been run on real Binance data, and the order-placement code is untested.

2. Pipeline

Step	What it does
1. Daily bias	Three votes (+1 / -1): A previous-day high/low, B daily swing structure (HH+HL / LH+LL), C yesterday's liquidity sweep. Trade only when $ \text{score} \geq 2$. Locked once at 00:00 UTC. Buys only on bullish days, sells only on bearish days, nothing on neutral days.
2. 15m sweep	Wick beyond the lowest low (bullish) or highest high (bearish) of the previous 24 15m candles, with a close back inside.
3. 5m IFVG	Track FVGs of the opposite type. A candle <i>body</i> closing through the gap inverts it (wicks do not count). Inversion must come after the sweep candle closes.
4. Entry	Retest of the inverted zone with a rejection candle closing in the bias direction. Entry at that candle's close. Zone is dropped if price closes through it.
5. Risk	Stop beyond the sweep extreme (0.05% buffer). Target 1:3 R:R. One trade per day.

3. Backtest mode

```
python ict_fvg_bot.py backtest --symbol BTCUSDT --start 2025-01-01 --end 2026-01-01
```

Runs the same pipeline gated by each bias method alone (A, B, C) and by the combined score, and reports bias days, direction accuracy, trades, win rate, total R and average R per variant, then lists the combined-score trades. If stop and target are hit in the same candle, the stop is counted.

4. Commands

Command	Purpose
<code>backtest</code>	Historical klines; per-method and combined comparison. <code>--loose-body</code> , <code>--rr</code> , <code>--fee-r</code> available.
<code>signal</code>	One-shot: prints today's votes, bias and any setups.
<code>live --testnet</code>	Polls every 30s, locks bias daily, logs signals. Orders only with <code>--execute</code> (market + <code>STOP_MARKET</code> + <code>TAKE_PROFIT_MARKET</code>). Needs <code>BINANCE_API_KEY</code> / <code>BINANCE_API_SECRET</code> .

5. Choices to check

- **Method C:** the spec's `close > d[-2].low` looked wrong; implemented as 'closed back above the swept level'.
- **Body fully through:** implemented as the body spanning the whole gap (open at or before the near edge, close beyond the far edge). May give few trades; `--loose-body` only needs a close beyond the far edge.
- **Before real funds:** backtest on real klines, then testnet dry-run, then `--execute` on testnet.

Appendix: source (ict_fvg_bot.py)

```
#!/usr/bin/env python3
"""ICT bot for Binance USD-M futures (stdlib only).

Pipeline: daily bias -> 15m liquidity sweep -> 5m IFVG retest -> entry (1:3 R:R).

python ict_fvg_bot.py backtest --symbol BTCUSDT --start 2025-01-01 --end 2026-01-01
python ict_fvg_bot.py signal --symbol BTCUSDT # one-shot, prints today's state
python ict_fvg_bot.py live --symbol BTCUSDT --testnet # dry-run unless --execute

Daily bias is locked once at 00:00 UTC. Never trades against it.
"""
from __future__ import annotations

import argparse
import hashlib
import hmac
import json
import os
import time
import urllib.parse
import urllib.request
from bisect import bisect_left
from dataclasses import dataclass, field
from datetime import datetime, timezone

MIN = 60_000
DAY = 86_400_000
LIVE_URL = "https://fapi.binance.com"
TESTNET_URL = "https://testnet.binancefuture.com"

BULL, BEAR = 1, -1

@dataclass
class Candle:
    t: int # open time, ms
    o: float
    h: float
    l: float
    c: float

# ----- data
def _get(base, path, params=None, headers=None):
    url = base + path + ("?" + urllib.parse.urlencode(params) if params else "")
    req = urllib.request.Request(url, headers=headers or {})
    with urllib.request.urlopen(req, timeout=20) as r:
        return json.load(r)

def fetch_klines(symbol, interval, start_ms, end_ms, base=LIVE_URL):
    out, cur = [], start_ms
    while cur < end_ms:
        rows = _get(base, "/fapi/v1/klines", {"symbol": symbol, "interval": interval,
                                              "startTime": cur, "endTime": end_ms, "limit": 1500})
        if not rows:
            break
        out += [Candle(r[0], float(r[1]), float(r[2]), float(r[3]), float(r[4])) for r in rows]
        cur = rows[-1][0] + 1
        if len(rows) < 1500:
            break
    return out

_T = {}

def times(cs):
    """Cached list of open times for a candle list (used with bisect)."""
    e = _T.get(id(cs))
    if e is None or e[0] is not cs or e[1] != len(cs):
        e = _T[id(cs)] = (cs, len(cs), [c.t for c in cs])
    return e[2]

# ----- swings / FVG logic
def swing_highs(cs, n=2):
    """Indices of fractal swing highs (n lower-or-equal-high candles each side, strict vs. left)."""
    return [i for i in range(n, len(cs) - n)
            if all(cs[i].h > cs[i - k].h and cs[i].h >= cs[i + k].h for k in range(1, n + 1))]
```

```

def swing_lows(cs, n=2):
    return [i for i in range(n, len(cs) - n)
            if all(cs[i].l < cs[i - k].l and cs[i].l <= cs[i + k].l for k in range(1, n + 1))]

@dataclass
class Zone:
    kind: int          # BULL / BEAR (FVG type, flips to the opposite when inverted)
    low: float
    high: float
    formed: int         # index of candle completing the 3-candle pattern
    inverted_at: int = -1

def fvg_at(cs, i):
    """FVG completed by candle i (3-candle pattern i-2, i-1, i), or None."""
    if i < 2:
        return None
    if cs[i].l > cs[i - 2].h:
        return Zone(BULL, cs[i - 2].h, cs[i].l, i)
    if cs[i].h < cs[i - 2].l:
        return Zone(BEAR, cs[i].h, cs[i - 2].l, i)
    return None

def inverts(zone, c, strict=True):
    """True if candle c closes through the gap with its body (wicks don't count).

    strict: body spans the whole gap (open on the near side, close beyond the far side).
    non-strict: just a close beyond the far side.
    Bearish FVG closed up through -> bullish IFVG; bullish FVG closed down through -> bearish IFVG.
    """
    if zone.kind == BEAR:
        return c.c > zone.high and (not strict or c.o <= zone.low)
    return c.c < zone.low and (not strict or c.o >= zone.high)

# ----- daily bias
def vote_pdh_pdl(d, price):
    """A. price above previous-day high +1, below previous-day low -1."""
    return BULL if price > d[-1].h else BEAR if price < d[-1].l else 0

def vote_structure(d, n=2):
    """B. HH+HL -> +1, LH+LL -> -1 from the last two confirmed daily swings."""
    hi, lo = swing_highs(d, n)[-2:], swing_lows(d, n)[-2:]
    if len(hi) < 2 or len(lo) < 2:
        return 0
    hh, hl = d[hi[1]].h > d[hi[0]].h, d[lo[1]].l > d[lo[0]].l
    lh, ll = d[hi[1]].h < d[hi[0]].h, d[lo[1]].l < d[lo[0]].l
    return BULL if hh and hl else BEAR if lh and ll else 0

def vote_sweep(d, lookback=5):
    """C. Yesterday swept a major low and closed back above it +1; swept a major high and closed below -1."""
    if len(d) < lookback + 1:
        return 0
    prev, y = d[-lookback - 1:-1], d[-1]
    lvl_lo, lvl_hi = min(c.l for c in prev), max(c.h for c in prev)
    if y.l < lvl_lo and y.c > lvl_lo:
        return BULL
    if y.h > lvl_hi and y.c < lvl_hi:
        return BEAR
    return 0

METHODS = {"A_pdh_pdl": lambda d, p: vote_pdh_pdl(d, p),
           "B_structure": lambda d, p: vote_structure(d),
           "C_sweep": lambda d, p: vote_sweep(d)}

def daily_bias_votes(d, price):
    return {k: f(d, price) for k, f in METHODS.items()}

def daily_bias(d, price, min_score=2):
    """d = completed daily candles (d[-1] = yesterday); price = price at the daily open."""
    score = sum(daily_bias_votes(d, price).values())
    return BULL if score >= min_score else BEAR if score <= -min_score else 0

# ----- 15m sweep -> 5m IFVG entry

```

```

@dataclass
class Setup:
    direction: int
    entry_t: int      # open time of the entry (rejection) 5m candle
    entry: float      # its close
    stop: float
    target: float
    sweep_extreme: float
    sweep_t: int
    zone: Zone = field(repr=False, default=None)

@dataclass
class Params:
    sweep_lookback: int = 24      # 15m candles defining the "recent" low/high
    fvg_wait: int = 48            # 5m candles after the sweep to find the IFVG + retest
    rr: float = 3.0
    stop_buffer: float = 0.0005  # fraction of price beyond the sweep extreme
    strict_body: bool = True
    max_trades_per_day: int = 1

def find_sweeps(c15, k0, k1, direction, p):
    """Yield (index, level, extreme) of 15m sweeps in c15[k0:k1] in the bias direction."""
    for k in range(max(k0, p.sweep_lookback), k1):
        win = c15[k - p.sweep_lookback:k]
        c = c15[k]
        if direction == BULL:
            lvl = min(x.l for x in win)
            if c.l < lvl and c.c > lvl:
                yield k, lvl, c.l
        else:
            lvl = max(x.h for x in win)
            if c.h > lvl and c.c < lvl:
                yield k, lvl, c.h

def find_ifvg_entry(c5, i0, i1, direction, sweep_t, extreme, p):
    """Scan 5m candles [i0, i1) after a sweep: FVG -> body-close inversion -> retest rejection."""
    sweep_close = sweep_t + 15 * MIN
    wanted = BEAR if direction == BULL else BULL      # FVG type that inverts into our direction
    active, inverted = [], []
    for j in range(i0, min(i1, i0 + p.fvg_wait)):
        c = c5[j]
        # 1. retest of an already-inverted zone (must be a later candle than the inversion)
        for z in inverted[:]:
            if direction == BULL:
                if c.c < z.low:
                    inverted.remove(z); continue      # IFVG failed
                if c.l <= z.high and c.c > z.high and c.c > c.o:
                    return c, z
            else:
                if c.c > z.high:
                    inverted.remove(z); continue
                if c.h >= z.low and c.c < z.low and c.c < c.o:
                    return c, z
        # 2. inversion: only after the sweep candle has closed
        if c.t >= sweep_close:
            for z in active[:]:
                if inverts(z, c, p.strict_body):
                    active.remove(z); z.inverted_at = j; inverted.append(z)
        # 3. new FVG; its first candle must not predate the sweep candle
        z = fvg_at(c5, j)
        if z and z.kind == wanted and c5[j - 2].t >= sweep_t:
            active.append(z)
    return None

def find_setups(c15, c5, day_start, direction, p):
    """Setups for one day, earliest first. direction = locked daily bias."""
    if direction == 0:
        return []
    day_end = day_start + DAY
    k0 = bisect_left(times(c15), day_start)
    k1 = bisect_left(times(c15), day_end)
    t5 = times(c5)
    i_end = bisect_left(t5, day_end)
    out, last_entry_t = [], -1
    for k, lvl, extreme in find_sweeps(c15, k0, k1, direction, p):
        sweep_t = c15[k].t
        i0 = bisect_left(t5, sweep_t)      # include sweep-window candles for FVG formation
        hit = find_ifvg_entry(c5, i0, i_end, direction, sweep_t, extreme, p)

```

```

        if not hit:
            continue
        c, z = hit
        if c.t <= last_entry_t:
            continue
        entry = c.c
        stop = extreme * (1 - p.stop_buffer) if direction == BULL else extreme * (1 + p.stop_buffer)
        risk = (entry - stop) * direction
        if risk <= 0:
            continue
        out.append(Setup(direction, c.t, entry, stop, entry + direction * p.rr * risk, extreme, sweep_t, z))
        last_entry_t = c.t
    out.sort(key=lambda s: s.entry_t)
    return out

def simulate(setup, c5, day_end):
    """Walk 5m candles after entry. Returns (R multiple, exit reason). Same-candle SL+TP -> SL (conservative)."""
    d = setup.direction
    risk = (setup.entry - setup.stop) * d
    last = None
    for c in c5[bisect_left(times(c5), setup.entry_t + 5 * MIN):]:
        if c.t >= day_end:
            break
        last = c
        hit_sl = c.l <= setup.stop if d == BULL else c.h >= setup.stop
        hit_tp = c.h >= setup.target if d == BULL else c.l <= setup.target
        if hit_sl:
            return -1.0, "SL"
        if hit_tp:
            return (setup.target - setup.entry) * d / risk, "TP"
    if last is None:
        return 0.0, "NOFILL"
    return (last.c - setup.entry) * d / risk, "EOD"

# ----- backtest
def backtest(daily, c15, c5, p, start_ms=0, warmup=12, fee_r=0.0):
    """Runs the same pipeline gated by each bias method alone and by the combined score."""
    variants = list(METHODS) + ["combined"]
    stats = {v: dict(days=0, trades=0, wins=0, r=0.0, bias_days=0, bias_right=0) for v in variants}
    log = []
    for i in range(warmup, len(daily)):
        if daily[i].t < start_ms:
            continue
        d_hist, today = daily[:i], daily[i]
        price = today.o # bias locked at the 00:00 UTC open
        votes = daily_bias_votes(d_hist, price)
        biases = dict(votes)
        biases["combined"] = daily_bias(d_hist, price)
        actual = BULL if today.c > today.o else BEAR
        for v in variants:
            b, s = biases[v], stats[v]
            s["days"] += 1
            if b == 0:
                continue
            s["bias_days"] += 1
            s["bias_right"] += (b == actual)
            n = 0
            for su in find_setups(c15, c5, today.t, b, p):
                if n >= p.max_trades_per_day:
                    break
                r, why = simulate(su, c5, today.t + DAY)
                r -= fee_r
                s["trades"] += 1; s["wins"] += r > 0; s["r"] += r; n += 1
                if v == "combined":
                    log.append((datetime.fromtimestamp(su.entry_t / 1000, timezone.utc), b, su.entry, su.stop, su.target, why))
    return stats, log

def print_report(stats, log):
    print(f"{'bias method':<14}{'bias days':>10}{'dir acc':>9}{'trades':>8}{'win%':>7}{'total R':>9}{'avg R':>8}")
    for v, s in stats.items():
        acc = s["bias_right"] / s["bias_days"] * 100 if s["bias_days"] else 0
        wr = s["wins"] / s["trades"] * 100 if s["trades"] else 0
        avg = s["r"] / s["trades"] if s["trades"] else 0
        print(f"{v:<14}{s['bias_days']:>10}{acc:>8.1f}%{s['trades']:>8}{wr:>6.1f}%{s['r']:>9.2f}{avg:>8.2f}")
    print("\ncombined-score trades:")
    for t, b, e, sl, tp, why, r in log:
        print(f" {t:%Y-%m-%d %H:%M} {'LONG ' if b == BULL else 'SHORT'} entry={e:.2f} sl={sl:.2f} tp={tp:.2f} {why:<5} {r:+.2f}")
# ----- live
class Binance:

```

```

def __init__(self, testnet):
    self.base = TESTNET_URL if testnet else LIVE_URL
    self.key, self.secret = os.environ.get("BINANCE_API_KEY", ""), os.environ.get("BINANCE_API_SECRET", "")

def signed(self, method, path, params):
    params = dict(params, timestamp=int(time.time() * 1000), recvWindow=5000)
    q = urllib.parse.urlencode(params)
    sig = hmac.new(self.secret.encode(), q.encode(), hashlib.sha256).hexdigest()
    req = urllib.request.Request(f"{self.base}{path}?{q}&signature={sig}", method=method,
                                headers={"X-MBX-APIKEY": self.key})
    with urllib.request.urlopen(req, timeout=20) as r:
        return json.load(r)

def step_size(self, symbol):
    info = _get(self.base, "/fapi/v1/exchangeInfo")
    for s in info["symbols"]:
        if s["symbol"] == symbol:
            f = {x["filterType"]: x for x in s["filters"]}["LOT_SIZE"]
            return float(f["stepSize"])
    raise ValueError(symbol)

def place(self, symbol, su, risk_usdt):
    qty = risk_usdt / abs(su.entry - su.stop)
    step = self.step_size(symbol)
    qty = int(qty / step) * step
    side, opp = ("BUY", "SELL") if su.direction == BULL else ("SELL", "BUY")
    common = dict(symbol=symbol, quantity=f"{qty:.8f}".rstrip("0"))
    self.signed("POST", "/fapi/v1/order", dict(common, side=side, type="MARKET"))
    self.signed("POST", "/fapi/v1/order", dict(common, side=opp, type="STOP_MARKET",
                                              stopPrice=f"{su.stop:.2f}", reduceOnly="true"))
    self.signed("POST", "/fapi/v1/order", dict(common, side=opp, type="TAKE_PROFIT_MARKET",
                                              stopPrice=f"{su.target:.2f}", reduceOnly="true"))

def live_state(symbol, p, base):
    now = int(time.time() * 1000)
    day_start = now - now % DAY
    daily = fetch_klines(symbol, "1d", day_start - 40 * DAY, day_start - 1, base) # completed days only
    c15 = fetch_klines(symbol, "15m", day_start - 2 * DAY, now, base)
    c5 = fetch_klines(symbol, "5m", day_start - 2 * DAY, now, base)
    c15 = [c for c in c15 if c.t + 15 * MIN <= now] # drop forming candles
    c5 = [c for c in c5 if c.t + 5 * MIN <= now]
    day_open = next(c.o for c in c5 if c.t >= day_start)
    return daily, c15, c5, day_start, day_open

def run_live(args):
    p = Params()
    api = Binance(args.testnet)
    locked, traded = {}, set()
    while True:
        daily, c15, c5, day_start, day_open = live_state(args.symbol, p, api.base)
        if day_start not in locked: # lock once per day, at the first poll after 00:00 UTC
            locked[day_start] = daily_bias(daily, day_open)
            print(f"[{datetime.fromtimestamp(day_start / 1000, timezone.utc):%Y-%m-%d}] bias locked: "
                  f"{ {'BULL': 'BULLISH', 'BEAR': 'BEARISH', 0: 'NEUTRAL'}[locked[day_start]] } {daily_bias_votes(daily, day_open)}")
        for su in find_setups(c15, c5, day_start, locked[day_start], p)[0:p.max_trades_per_day]:
            if su.entry_t in traded or su.entry_t < int(time.time() * 1000) - 10 * MIN:
                continue # already handled / stale
            traded.add(su.entry_t)
            print("SIGNAL", su)
            if args.execute:
                api.place(args.symbol, su, args.risk_usdt)
            time.sleep(30)

# ----- CLI
def ts(s):
    return int(datetime.strptime(s, "%Y-%m-%d").replace(tzinfo=timezone.utc).timestamp() * 1000)

def main():
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
    sub = ap.add_subparsers(dest="cmd", required=True)
    b = sub.add_parser("backtest")
    b.add_argument("--symbol", default="BTCUSDT"); b.add_argument("--start", required=True); b.add_argument("--end", required=True)
    b.add_argument("--rr", type=float, default=3.0); b.add_argument("--loose-body", action="store_true", help="IFVG needs onl")
    b.add_argument("--fee-r", type=float, default=0.0, help="cost per trade in R")
    s = sub.add_parser("signal"); s.add_argument("--symbol", default="BTCUSDT"); s.add_argument("--testnet", action="store_tr")
    l = sub.add_parser("live"); l.add_argument("--symbol", default="BTCUSDT"); l.add_argument("--testnet", action="store_true")
    l.add_argument("--execute", action="store_true", help="place orders (default: log signals only)")
    l.add_argument("--risk-usdt", type=float, default=10.0)

```

```

a = ap.parse_args()

if a.cmd == "backtest":
    p = Params(rr=a.rr, strict_body=not a.loose_body)
    s0, e0 = ts(a.start), ts(a.end)
    daily = fetch_klines(a.symbol, "1d", s0 - 30 * DAY, e0)
    c15 = fetch_klines(a.symbol, "15m", s0 - 31 * DAY, e0)
    c5 = fetch_klines(a.symbol, "5m", s0 - 31 * DAY, e0)
    stats, log = backtest(daily, c15, c5, p, start_ms=s0, fee_r=a.fee_r)
    print_report(stats, log)
elif a.cmd == "signal":
    p = Params()
    base = TESTNET_URL if a.testnet else LIVE_URL
    daily, c15, c5, day_start, day_open = live_state(a.symbol, p, base)
    b = daily_bias(daily, day_open)
    print("votes:", daily_bias_votes(daily, day_open), "->", {BULL: "BULLISH", BEAR: "BEARISH", 0: "NEUTRAL"}[b])
    for su in find_setups(c15, c5, day_start, b, p):
        print(su)
else:
    run_live(a)

if __name__ == "__main__":
    main()

```